Create head pose estimation dataset

In [ ]:
```
# !pip install tensorflow-datasets

import os
import multiprocessing
import absl
import numpy as np

from pathlib import Path
from PIL import Image

import tensorflow as tf
from tensorflow.python.ops import control_flow_ops
import tensorflow_datasets as tfds

logger = absl.logging
```

In [ ]:
```
def pad_square(image, target_h, target_w):
  h, w = image.shape[0:2]
  y = np.abs(target_h - h) // 2
  x = np.abs(target_w - w) // 2
  new_image = np.pad(image, ((y, target_h - h - y), (x, target_w - w - x), (0, 0)))
  return new_image

def resize_and_pad_image(image, size):
  h, w = image.shape[0:2]
  max_side = max(h, w)
  new_h = int(size / max_side * h)
  new_w = int(size / max_side * w)
  im = Image.fromarray(image)
  im = im.resize((new_w, new_h), Image.LANCZOS)
  im = pad_square(np.array(im), size, size)
  return im

def resize_func(image, bbox, pose):
  image = Image.fromarray(image.numpy()).convert('RGB')
  bbox = bbox.numpy()
  pose = pose.numpy()

  # Crop
  w, h = image.size
  ymin, xmin, ymax, xmax = bbox
  box_w = abs(xmax - xmin)
  box_h = abs(ymax - ymin)

  # 0.1 ~ 0.5
  random_scales = np.array([0.5])
  xmin = max(0, xmin - box_w * np.random.choice(random_scales))
  xmax = min(w, xmax + box_w * np.random.choice(random_scales))
  ymin = max(0, ymin - box_h * np.random.choice(random_scales))
  ymax = min(h, ymax + box_h * np.random.choice(random_scales))
  image = image.crop([int(xmin), int(ymin), int(xmax), int(ymax)])

  image = np.array(image, dtype=np.uint8)
  image = resize_and_pad_image(image, 128)
  return image, bbox, pose

def read_example(example):
  image = example['image']
  landmarks_2d = example['landmarks_2d']
  pose = example['pose_params']

  x = tf.expand_dims(landmarks_2d[:, 0], 0)
  y = tf.expand_dims(landmarks_2d[:, 1], 0)
  xmin, xmax = tf.math.reduce_min(x), tf.math.reduce_max(x)
  ymin, ymax = tf.math.reduce_min(y), tf.math.reduce_max(y)
  bbox = tf.stack([ymin, xmin, ymax, xmax]) * 450.0

  image, bbox, pose = tf.py_function(
    resize_func, [image, bbox, pose], (tf.uint8, tf.int32, tf.float32))
  pose = pose[:3] * 180 / np.pi
  return image, pose
```

In [ ]:
```
dataset = tfds.load('the300w_lp', split='train')
dataset = dataset.map(read_example)
dataset = dataset.prefetch(buffer_size=tf.data.experimental.AUTOTUNE)
```

In [ ]:
```
data, label = [], []
for e in dataset.take(20000):
  data.append(e[0].numpy())
  label.append(e[1].numpy())
np.save('data_20000.npy', np.array(data))
np.save('label_20000.npy', np.array(label))
```

In [ ]:
```
with open('data_20000.npy', 'rb') as data, open('label_20000.npy', 'rb') as label:
  data = np.load(data)
  print(data.shape)
  label = np.load(label)
  print(label.shape)
```